

Nama : Derren Roosevelt Wijaya

NPM : 252310029

Kelas : TI – 25 – PA2

TUGAS 12

Link Github : https://github.com/wijayaderren2-prog/232510029_LabAlgoritma.git

```
Shortest distance from node 1 to node 5 = 11
Path: 1 -> 3 -> 2 -> 4 -> 5
```

```
-----
Process exited after 0.3555 seconds with return value 0
Press any key to continue . . . |
```

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  const int INF = 1e9;
8
9  void printPath(int current_node, const vector<int>& parent, int start_node) {
10     if (current_node == -1) {
11         return;
12     }
13
14     printPath(parent[current_node], parent, start_node);
15
16     if (current_node != start_node) {
17         cout << " -> ";
18     }
19     cout << current_node;
20 }
21
22 void dijkstra(int start_node, int target_node, int num_vertices, const vector<vector<pair<int, int>>>& adj) {
23     vector<int> dist(num_vertices + 1, INF);
24     dist[start_node] = 0;
25
26     vector<int> parent(num_vertices + 1, -1);
27
28     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
29     pq.push(make_pair(0, start_node));
30
31     while (!pq.empty()) {
32         int current_dist = pq.top().first;
33         int current_node = pq.top().second;
34         pq.pop();
35
36         if (current_dist > dist[current_node]) continue;
37     }
```

```

42     if (dist[current_node] + weight < dist[neighbor_node]) {
43         dist[neighbor_node] = dist[current_node] + weight;
44         parent[neighbor_node] = current_node;
45         pq.push(make_pair(dist[neighbor_node], neighbor_node));
46     }
47 }
48
49
50 if (dist[target_node] == INF) {
51     cout << "Tidak ada jalur dari node " << start_node << " ke node " << target_node << endl;
52 } else {
53     cout << "Shortest distance from node " << start_node << " to node " << target_node << " = " << dist[target_node] << endl;
54     cout << "Path: ";
55     printPath(target_node, parent, start_node);
56     cout << endl;
57 }
58
59
60 int main() {
61     int num_vertices = 5;
62
63     vector<vector<pair<int, int> > > adj(num_vertices + 1);
64
65     adj[1].push_back(make_pair(2, 4));
66     adj[1].push_back(make_pair(3, 1));
67     adj[3].push_back(make_pair(2, 2));
68     adj[2].push_back(make_pair(4, 5));
69     adj[3].push_back(make_pair(4, 8));
70     adj[4].push_back(make_pair(5, 3));
71
72     int start_node = 1;
73     int target_node = 5;
74
75     dijkstra(start_node, target_node, num_vertices, adj);
76
77     return 0;
78 }

```